

UNIVERSIDAD DE GUADALAJARA
CENTRO UNIVERSITARIO DE TONALA
PROGRAMACION DE SERVICIOS WEB



Alumno:

Osmar Daniel Campechano Garcia

Braulio Ivan Hernández Fernández

Miguel Angel Nicasio Medina

Alan Fernando Ramos Aguilar

Profesor:

José Navarro

- **¿Qué es un servicio web?**

Un servicio web es un componente de software de Internet diseñado para facilitar la transferencia de datos y la comunicación entre aplicaciones a través de una red.

Dado que un servicio web expone los datos y la funcionalidad de una aplicación para que otros sistemas la utilicen, **cada servicio web es una API, pero no todas las API son servicios web.**

Las API en general actúan como intermediarias entre aplicaciones desconectadas; en cambio, los servicios web **requieren obligatoriamente el uso de una red** para lograr esa conexión.

- **¿Qué es una API?**

Una **API** (*Application Programming Interface* o **interfaz de programación de aplicaciones**) es un conjunto de reglas o protocolos que permiten a las aplicaciones de software comunicarse entre sí para intercambiar datos, características y funcionalidades.

Actúa como un **intermediario o puente** entre aplicaciones desconectadas, facilitando la comunicación a través de un modelo de solicitud (enviada por una aplicación cliente) y respuesta (proporcionada por un servidor)

- **Diferencias entre un servicio web y una API.**

Las principales diferencias entre un **servicio web** y una **API** se centran en su alcance, el uso de red y sus modelos de acceso:

Relación de inclusión: **Cada servicio web es una API, pero no todas las API son servicios web.** Los servicios web constituyen una categoría o subconjunto específico dentro del concepto más amplio de API.

Requisito de red: Un servicio web **requiere obligatoriamente el uso de una red** para facilitar la conexión y la transferencia de datos entre aplicaciones. En cambio, las API pueden funcionar a través de una red (API web o remotas) o de manera local dentro de un mismo equipo (como las API del sistema operativo o de bases de datos).

Mecanismo de interacción: Aunque los formatos de intercambio de datos varíen según el servicio web empleado, las solicitudes y respuestas de información entre aplicaciones se gestionan siempre a través de una API.

Modelos de acceso y privacidad: Los servicios web suelen ser privados y restringir su acceso únicamente a usuarios o sistemas previamente autorizados. Por el contrario, las API cubren un rango más amplio de acceso, clasificándose en abiertas/públicas, de socios, internas o compuestas.

- Arquitectura cliente-servidor.

La **arquitectura cliente-servidor** es un modelo de comunicación centrado en el intercambio de **solicitudes y respuestas** entre dos partes principales:

- **Cliente:** Es la aplicación que envía la solicitud. En el desarrollo web y móvil, las aplicaciones front-end actúan como clientes que se comunican con un backend para descargar o cargar datos y acceder a servicios.
- **Servidor:** Es el programa o sistema remoto que recibe la solicitud, la procesa y proporciona la respuesta requerida.
- Métodos HTTP (GET, POST, PUT, PATCH y DELETE).

Los métodos o verbos HTTP definen las acciones que una aplicación cliente solicita ejecutar sobre los recursos de un servidor web, correspondiéndose con las operaciones CRUD (Crear, Leer, Actualizar y Eliminar).

1. GET (Lectura/Consulta): Se utiliza para recuperar datos del servidor. Es un método seguro e idempotente, lo que significa que no modifica el estado ni los datos del recurso.
2. POST (Creación): Envía datos al servidor para crear un nuevo recurso o procesar información. Cada ejecución puede generar un nuevo registro (no es idempotente).
3. PUT vs. PATCH (Actualización): PUT se utiliza para reemplazar por completo un recurso existente (o crearlo si se especifica la ubicación exacta) y es idempotente. PATCH se emplea para aplicar modificaciones o actualizaciones parciales sobre un recurso.
4. DELETE (Eliminación): Elimina el recurso especificado identificado en la URI de la solicitud.

5. Idempotencia y Seguridad: Los métodos GET, PUT y DELETE son idempotentes porque realizar la misma solicitud varias veces produce el mismo resultado final en el servidor.

- Códigos de estado HTTP más utilizados.

Los códigos de estado HTTP son respuestas numéricas de tres dígitos emitidas por el servidor que informan al cliente sobre el resultado de una solicitud, organizándose en cinco familias según el primer dígito.

Aspectos clave identificados:

1. **2xx (Respuestas satisfactorias / Éxito)**: Indican que la solicitud fue recibida, entendida y aceptada correctamente. Los más utilizados son **200 OK** (solicitud correcta), **201 Created** (un recurso fue creado con éxito) y **204 No Content** (éxito sin contenido devuelto en el cuerpo).
2. **3xx (Redirecciones)**: Indican que el cliente debe realizar acciones adicionales para completar la petición. Los más comunes son **301 Moved Permanently** (el recurso se movió de forma definitiva) y **302 Found** (redirección temporal).
3. **4xx (Errores del cliente)**: Señalan que la solicitud contiene sintaxis incorrecta o no puede ser procesada por un problema originado en el cliente. Destacan **400 Bad Request** (solicitud mal formada), **401 Unauthorized** (requiere autenticación), **403 Forbidden** (acceso denegado por falta de permisos) y **404 Not Found** (el recurso solicitado no existe).
4. **5xx (Errores del servidor)**: Advierten que el servidor falló al intentar cumplir una solicitud válida. Entre los más habituales se encuentran **500 Internal Server Error** (error no especificado en el servidor), **502 Bad Gateway** (respuesta no válida enviada por un servidor intermedio o proxy) y **503 Service Unavailable** (servidor no disponible por sobrecarga o mantenimiento).

- Formatos de intercambio de datos (JSON y XML).

JSON y **XML** son los formatos estandarizados de intercambio de datos más representativos para la transferencia de información entre aplicaciones a través de servicios web y API:

1. JSON (JavaScript Object Notation)

- **Estructura y ligereza**: Es un formato ligero para el intercambio de datos que destaca por ser fácil de analizar.

- **Modelado de datos:** Utiliza pares de nombre/valor y listas ordenadas de valores.
- **Compatibilidad universal:** Debido a que emplea estructuras de datos universales, se puede utilizar con prácticamente cualquier lenguaje de programación.
- **Uso en API:** Se utiliza en llamadas a procedimientos remotos como JSON-RPC y es el formato más común en las API de estilo REST.

2. XML (eXtensible Markup Language)

- **Formato estructurado:** Protocolos como XML-RPC se basan en este formato específico para la transferencia de datos.
- **Requisito en SOAP:** El protocolo SOAP está basado en una especificación de mensajería que solo admite XML para el envío y recepción de información.

Soporte en arquitecturas de API

La elección del formato de intercambio varía según la arquitectura utilizada:

- **REST:** Los sistemas RESTful admiten múltiples formatos de mensajería para almacenar e intercambiar datos (entre ellos **JSON, XML, YAML, HTML y texto sin formato**), siendo esta flexibilidad una de las razones por las que REST es la opción predominante en API públicas.
- **SOAP:** Solo permite el uso de **XML**.

- Servicios REST y SOAP.

SOAP (Simple Object Access Protocol): Es un protocolo formal de mensajería basado en XML que permite la comunicación entre componentes de software que funcionan en entornos diferentes o están escritos en lenguajes distintos.

REST (Representational State Transfer): Es un estilo arquitectónico que establece un conjunto de principios y restricciones para la creación de API web.

SERVICIOS WEB Y API

CÓMO SE COMUNICAN LAS APLICACIONES

Conectando ideas, datos y personas en un mundo digital.



DATOS
APLICACIONES
SOLUCIONES
UN MUNDO
MÁS CONECTADO

¿Qué es un Servicio web?

Es un sistema que permite intercambiar datos o funciones entre aplicaciones mediante una red, utilizando estándares como HTTP.



Permite que diferentes sistemas se comuniquen, sin importar en qué plataforma estén.

¿Qué es una API?

Es un conjunto de reglas y puntos de acceso que permite a programas comunicarse entre sí, solicitando y recibiendo datos o funciones.



Una API define cómo se pueden solicitar los recursos y qué formato deben tener los datos.

Diferencia entre ambos

La API es el concepto amplio: puede ser local o remota. Un servicio web expone funciones por una red; es una forma de API.



Todos los servicios web son API, pero no todas las API son servicios web.

Arquitectura básica cliente-servidor



Ciclo de una petición HTTP



Métodos HTTP

GET	Consultar información.
POST	Crear un recurso / enviar datos.
PUT	Reemplazar un recurso completo.
PATCH	Modificar parcialmente un recurso.
DELETE	Eliminar un recurso.

Códigos de respuesta HTTP

200	OK – Solicitud exitosa.
201	Creado – Recurso creado correctamente.
400	Solicitud incorrecta – Datos inválidos.
401	No autenticado – Se requiere autenticación.
403	Prohibido – No tiene permisos.
404	No encontrado – El recurso no existe.
500	Error del servidor – Ocurrió un error en el servidor.

Formatos de intercambio de datos

JSON

```
{  
  "ciudad": "CDMX"  
}
```

JSON utiliza pares clave-valor, es ligero y muy utilizado en servicios web modernos.

XML

```
<ciudad>CDMX</ciudad>
```

XML utiliza etiquetas para estructurar la información y sigue un formato más estricto.

REST vs SOAP

REST

- Estilo arquitectónico.
- Usa recursos y HTTP.
- Suele usar JSON (puede usar otros formatos como XML, texto, etc.).

SOAP

- Protocolo.
- Mensajes estructurados en XML.
- Define un estándar más estricto.
- Suele usarse en aplicaciones empresariales.

Ejemplos reales de API o servicios web



Google Maps API

Integra mapas, ubicaciones, rutas y geocodificación en aplicaciones.

Se utiliza en apps de transporte, turismo, entregas, etc.



GitHub REST API

Permite consultar y gestionar repositorios, incidencias, usuarios y más.

Se utiliza en herramientas de desarrollo, automatizaciones y aplicaciones de terceros.



Spotify Web API

Permite consultar datos de música, artistas, álbumes y gestionar playlists.

Se utiliza en apps de música, análisis de datos y experiencias personalizadas.



Conclusión del equipo

Las API y los servicios web conectan sistemas distintos. Comprender HTTP, sus formatos y arquitecturas nos ayuda a desarrollar aplicaciones interoperables y seguras.



MÁS
IDEAS
MÁS
POSIBILIDADES

Referencias

Angular. (2025). Angular HTTP Client. <https://angular.dev/guide/http>

Express.js. (s.f.). Express Guide. <https://expressjs.com>

IBM. (2023). ¿Qué es una API?. <https://www.ibm.com/mx-es/topics/api>

Laravel. (2025). API Resources. <https://laravel.com/docs/12.x/eloquent-resources>

Mozilla Developer Network. (2025). HTTP Overview.
<https://developer.mozilla.org/docs/Web/HTTP/Overview>

OpenAPI Initiative. (2025). OpenAPI Specification. <https://www.openapis.org>

Postman. (2025). API Fundamentals Student Expert. <https://academy.postman.com>